

Restaurant Order Prediction and Visualization Dashboard: A Categorical Naive Bayes Approach

Rachith Bharadwaj T N (24BDS062), Ullas G M (24BDS085),
M G Shree Harsha (24BDS037)
Department of Data Science and Artificial Intelligence
Indian Institute of Information Technology Dharwad, India

Abstract—This paper presents a full-stack data handling and visualization system for predicting whether a user is likely to order restaurant food. The project transforms a structured dataset of 500 observations into an interactive decision-support dashboard. Four categorical factors—cuisine, time of day, weather, and hunger level—are encoded and modeled using a Categorical Naive Bayes classifier. The dataset is balanced with 250 “Yes” and 250 “No” outcomes and contains no missing values. The implemented prototype obtains 73.0% accuracy on the encoded dataset, with a macro F1-score of 0.73. The trained model and label encoders are serialized for reuse, exposed through a Flask REST API, and consumed by a Next.js frontend that presents class probabilities, confidence, feature distributions, and explanatory visualizations. The project demonstrates an end-to-end workflow connecting data preparation, probabilistic classification, backend integration, and interactive visual communication.

Index Terms—data handling, data visualization, restaurant order prediction, categorical Naive Bayes, Flask, Next.js, classification dashboard

I. INTRODUCTION

Restaurant ordering behavior depends on contextual factors that are naturally categorical. A customer may react differently to a cuisine at breakfast than at dinner, and weather or hunger level may alter the final decision. Converting these factors into a useful prediction requires more than training a classifier: the data must be inspected, encoded consistently, made available through an application interface, and communicated through understandable visualizations.

This project addresses that complete workflow. It predicts the binary target *OrderFood* from four inputs:

- 1) Cuisine: Chinese, Indian, Italian, or Mexican;
- 2) Time of day: Breakfast, Dinner, LateNight, or Lunch;
- 3) Weather: Cloudy, Rainy, or Sunny; and
- 4) Hunger level: High, Low, or Medium.

The principal contributions are:

- 1) a verified categorical dataset and exploratory visual analysis;
- 2) a reusable Categorical Naive Bayes prediction pipeline;
- 3) a Flask API for health checks and real-time predictions; and
- 4) a responsive Next.js dashboard for probability and feature visualization.

II. BACKGROUND AND RELATED TECHNOLOGIES

Naive Bayes classifiers apply Bayes’ theorem under a conditional-independence assumption. For a class y and feature vector $X = (x_1, \dots, x_n)$, the classifier estimates

$$P(y | X) \propto P(y) \prod_{i=1}^n P(x_i | y). \quad (1)$$

Categorical Naive Bayes is appropriate when every predictor takes one value from a finite set of categories [1]. The implementation uses Laplace smoothing to avoid zero probabilities for unseen feature-class combinations:

$$P(x_i = t | y = c) = \frac{N_{tic} + \alpha}{N_c + \alpha n_i}, \quad (2)$$

where N_{tic} is the count of category t for feature i in class c , N_c is the number of samples in class c , and n_i is the number of categories for feature i .

Pandas is used for tabular loading and grouped analysis [2]. Scikit-learn supplies the encoders, classifier, and evaluation metrics [1]. Flask provides lightweight HTTP endpoints [3], while the frontend uses the Next.js App Router and React components [4].

III. SYSTEM ARCHITECTURE

A. Overview

The implementation follows a three-layer architecture:

Data and Model Layer: A CSV dataset is loaded with Pandas. Categorical values are transformed using one label encoder per feature. A `CategoricalNB` model estimates the probability of the two output classes.

API Layer: A Flask service loads the serialized model and encoders, validates incoming JSON, applies the same encoding sequence, and returns a predicted label with class probabilities.

Presentation Layer: A Next.js 16 and React 19 application collects user selections and displays the prediction, confidence, probability meters, mathematical explanation, and feature-oriented visual analysis.

TABLE I
REST API ENDPOINTS

Endpoint	Method	Purpose
/api/health	GET	Reports service and model status
/api/predict	POST	Returns label, confidence, and class probabilities

TABLE II
DATASET ATTRIBUTES

Attribute	Levels	Categories
Cuisine	4	Chinese, Indian, Italian, Mexican
TimeOfDay	4	Breakfast, Dinner, LateNight, Lunch
Weather	3	Cloudy, Rainy, Sunny
HungerLevel	3	High, Low, Medium
OrderFood	2	No, Yes

B. Prediction Data Flow

The complete request flow is:

- 1) the user selects the four categorical inputs;
- 2) the frontend posts JSON to /api/predict;
- 3) the API validates and encodes the values;
- 4) the saved model computes $P(\text{No} | X)$ and $P(\text{Yes} | X)$;
- 5) the larger probability determines the class; and
- 6) the response is rendered as cards, bars, gauges, and summaries.

IV. DATASET HANDLING AND EXPLORATION

A. Dataset Profile

The dataset contains 500 rows and five columns. No field contains a missing value. The target is exactly balanced: 250 records indicate an order and 250 indicate no order. Table II summarizes the feature space.

B. Exploratory Findings

The strongest visible relationship is between hunger level and order decision. Among high-hunger records, 127 of 161 are labeled “Yes,” whereas low hunger produces 126 “No” outcomes out of 161. Rainy conditions contain 111 “Yes” outcomes compared with 63 “No” outcomes. Late-night observations also favor ordering, with 86 “Yes” and 50 “No” records. Cuisine is comparatively less separated across the two classes.

V. MODEL DEVELOPMENT

A. Encoding and Training

Separate `LabelEncoder` objects convert each feature to integer category indices. The target labels are encoded as `No=0` and `Yes=1`. The resulting 500×4 matrix is used to fit a Categorical Naive Bayes classifier. The trained classifier and all five encoders are persisted as `nb_model.pkl` and `encoders.pkl`, while conditional probability values are exported to JSON for analysis.

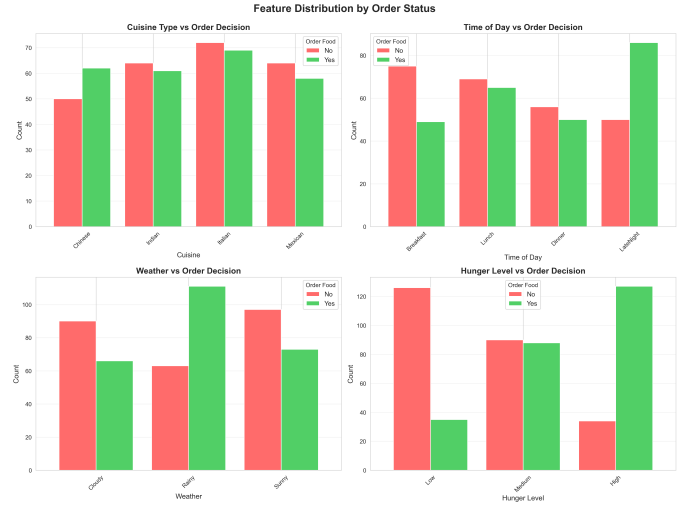


Fig. 1. Feature distributions grouped by restaurant order status.

TABLE III
PROTOTYPE MODEL PERFORMANCE

Metric	No	Yes	Overall
Precision	0.75	0.71	0.73
Recall	0.69	0.77	0.73
F1-score	0.72	0.74	0.73
Support	250	250	500
Accuracy	—	—	73.0%

B. Evaluation

The project notebook evaluates predictions over the complete encoded dataset. Therefore, the reported values describe prototype fit and should not be interpreted as an independent generalization estimate. A future revision should add a stratified train/test split and cross-validation.

The confusion matrix contains 173 correctly classified “No” records and 192 correctly classified “Yes” records. The model incorrectly predicts 77 “No” observations as “Yes” and 58 “Yes” observations as “No.” Its slightly higher recall for “Yes” indicates greater sensitivity to likely orders.

C. Feature Influence

The training notebook uses the variance of class-conditional log probabilities as an explanatory importance score. Hunger level has the largest value (0.2914), followed by weather (0.0421), time of day (0.0380), and cuisine (0.0105). This ranking agrees with the exploratory distributions: hunger level produces the clearest class separation.

VI. APPLICATION IMPLEMENTATION

A. Backend Service

The backend is implemented with Python, Flask, Flask-CORS, NumPy, Pandas, Joblib, and scikit-learn. At startup, it loads the classifier and encoders. The prediction endpoint checks for all required fields, encodes each value in the fixed

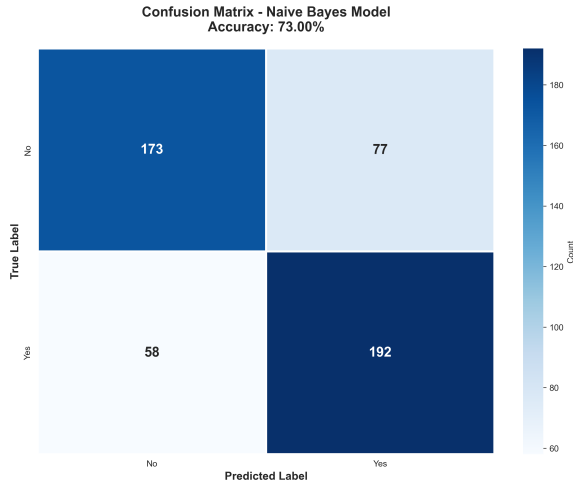


Fig. 2. Confusion matrix for the Categorical Naive Bayes model.

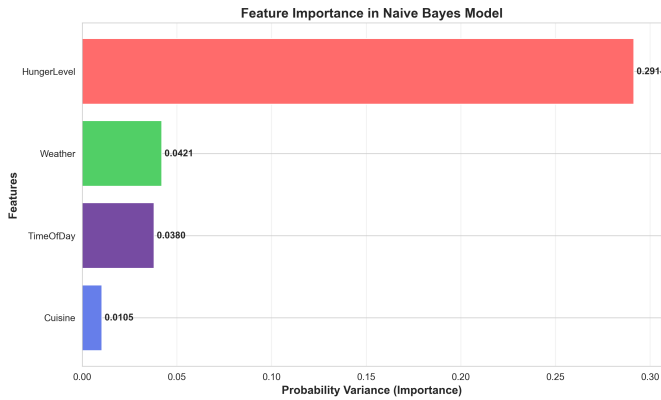


Fig. 3. Relative feature influence derived from conditional log-probability variance.

model order, generates the class probabilities, converts the result back to a human-readable label, and returns JSON.

For example, the stored model predicts “Yes” for Italian cuisine, lunch, sunny weather, and high hunger with 71.40% confidence. For Chinese cuisine, dinner, cloudy weather, and medium hunger, it predicts “No” with 55.68% confidence.

B. Frontend Dashboard

The frontend provides three main experiences:

- 1) a project landing page with team and system information;
- 2) a prediction playground for the four input factors; and
- 3) a mathematical model section explaining priors, likelihoods, Laplace smoothing, posterior probabilities, and final class selection.

Prediction results are shown through summary cards, circular probability views, percentage bars, a confidence gauge, and factor-oriented charts. This presentation supports the Data Handling and Visualization objective by connecting raw categories and model output to interpretable visual evidence.

VII. CHALLENGES AND LIMITATIONS

A. Categorical Consistency

The API accepts only categories learned by the saved encoders. Differences in spelling, capitalization, or feature order can produce inference errors. The implementation resolves this by using fixed option lists in the frontend and a fixed encoder order in the backend.

B. Model Interpretation

Naive Bayes assumes conditional independence among predictors. In practice, time of day, hunger, and ordering behavior may be correlated. The feature-importance visualization is an explanatory heuristic based on probability variance, not a causal measure.

C. Evaluation Scope

The current 73.0% accuracy is calculated on the same 500 records used for fitting. This is useful for validating the pipeline but may overestimate performance on unseen users. The current dataset is also small and synthetic or sample-oriented, so deployment decisions require larger, real-world data.

D. Prototype Paths and Deployment

Some backend notebook paths are machine-specific, and the Flask server uses the development configuration. A production version should use relative configuration, environment variables, automated tests, and a production WSGI server.

VIII. CONCLUSION AND FUTURE WORK

This project demonstrates a complete data handling and visualization pipeline for restaurant order prediction. It combines a balanced categorical dataset, exploratory plots, a Categorical Naive Bayes model, reusable serialized artifacts, a Flask prediction API, and a Next.js visualization dashboard. The verified prototype achieves 73.0% accuracy and shows that hunger level is the most influential observed factor in the current dataset.

Future work should:

- 1) use stratified train/test evaluation and k -fold cross-validation;
- 2) compare Naive Bayes with decision trees, random forests, and logistic regression;
- 3) collect larger real restaurant-order datasets with user and temporal context;
- 4) replace fixed explanatory weights in the API with model-derived values;
- 5) add API tests, error handling, and deployment configuration; and
- 6) extend the dashboard with downloadable analytical reports and longitudinal trends.

TEAM CONTRIBUTIONS

Rachith Bharadwaj T N worked on backend development and model integration. Ullas G M worked on frontend engineering and user-interface implementation. M G Shree Harsha worked on data visualization and system integration.

ACKNOWLEDGMENT

The authors thank the Department of Data Science and Artificial Intelligence, Indian Institute of Information Technology Dharwad, for supporting this Data Handling and Visualization project.

REFERENCES

- [1] Scikit-learn Developers, “Categorical Naive Bayes,” *Scikit-learn User Guide*. [Online]. Available: https://scikit-learn.org/stable/modules/naive_bayes.html
- [2] Pandas Development Team, “Categorical Data,” *Pandas User Guide*. [Online]. Available: https://pandas.pydata.org/docs/user_guide/categorical.html
- [3] Pallets Projects, “Flask Documentation,” [Online]. Available: <https://flask.palletsprojects.com/>
- [4] Vercel, “Next.js Documentation: App Router,” [Online]. Available: <https://nextjs.org/docs/app>
- [5] R. Bharadwaj T N, U. G. M., and M. G. Shree Harsha, “Data Handling and Visualization: Restaurant Order Prediction,” project repository. [Online]. Available: <https://github.com/Rachith000bharadwaj/Data-Handling-and-Visualization>